Київський національний університет імені Тараса Шевченка
Факультет комп'ютерних наук та кібернетики

Звіт
до лабораторної роботи №2

Виконала: студентка групи ДО-3
Булига А.І.

NoSQL бази даних виникли як відповідь на обмеження традиційних реляційних

баз (RDBMS), особливо коли йдеться про великі обсяги даних, необхідність швидкого масштабування та гнучкості схеми

В ході виконання роботи було розглянуто основні класифікації NoSQL рішень, які використовуються для вирішення специфічних задач, де реляційна модель є неефективною:

1. **Key-Value** (Ключ-Значення): (наприклад, Redis, DynamoDB). Найпростіша модель, що забезпечує надвисоку швидкість. Використовується для кешування, зберігання сесій користувачів та кошиків покупок
2. **Document-oriented** (Документоорієнтовані): (наприклад, MongoDB, CouchDB). Зберігають дані у форматі JSON/BSON. Дозволяють зберігати вкладені структури та масиви. Ідеальні для CMS, каталогів товарів, профілів користувачів
3. **Column-family** (Колонкові): (наприклад, Cassandra, HBase). Зберігають дані у вигляді колонок, а не рядків. Оптимізовані для запису та читання величезних масивів даних (Big Data, логи, метрики IoT)
4. **Graph** (Графові): (наприклад, Neo4j). Призначені для зберігання та обробки складних зв'язків між об'єктами (соціальні мережі, рекомендаційні системи, антифрод)

Для виконання лабораторної роботи було обрано документоорієнтовану СУБД **MongoDB**, оскільки вона найкраще підходить для роботи з ієрархічними даними предметної області (характеристики товарів та відгуки)

Враховуючи предметну область "Інтернет-магазин", було проаналізовано структуру даних та прийнято рішення розділити дані між двома типами сховищ

Дані, що залишаються в SQL (MS SQL Server):

- Сутності: Користувачі, Замовлення, Транзакції, Складський облік. Ці дані вимагають суворої узгодженості (ACID), підтримки транзакцій та чітких зв'язків (Relational Integrity). Фінансові втрати через неузгодженість даних неприпустимі

Дані, що виносяться в NoSQL (MongoDB):

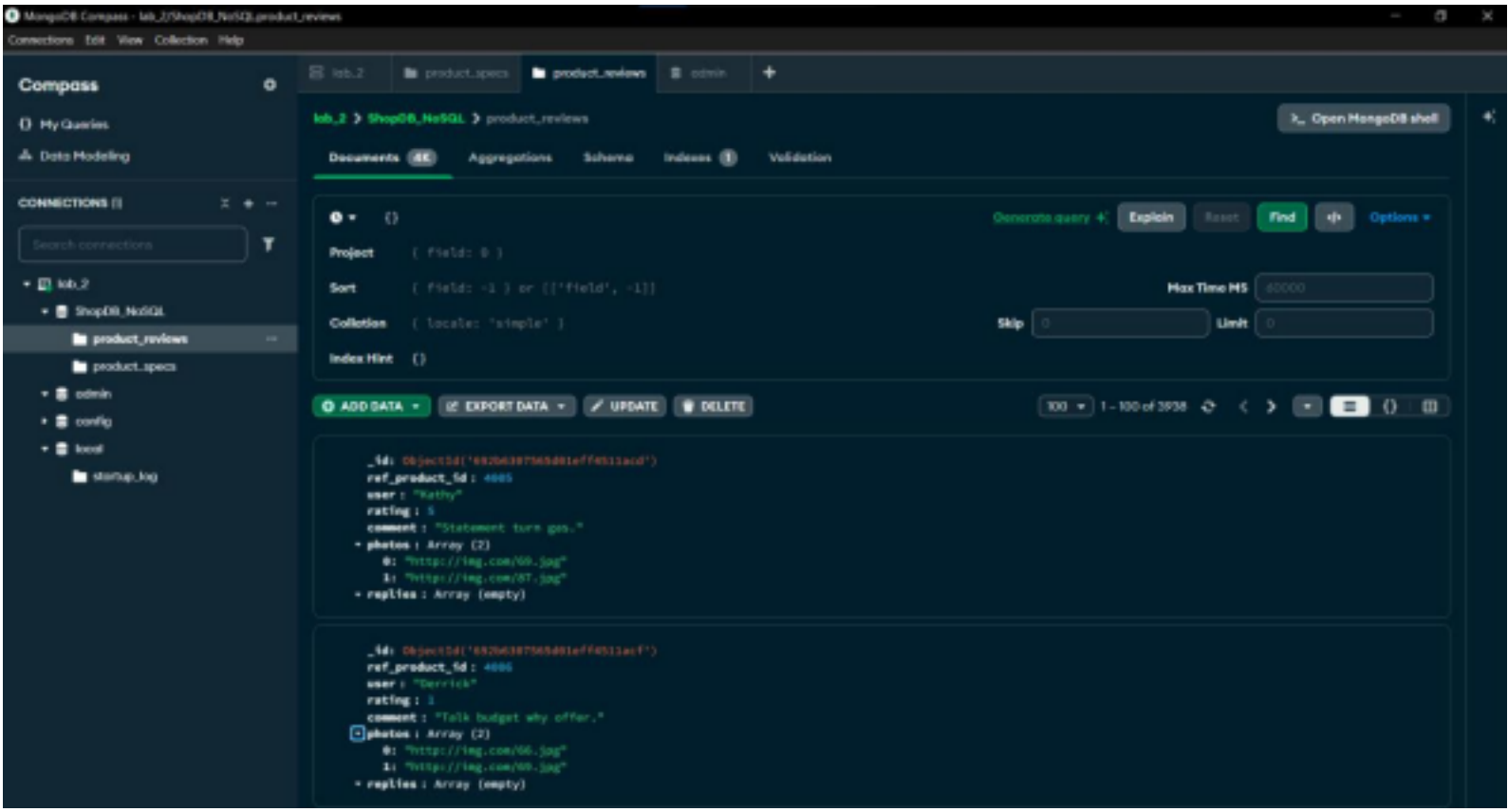
1. Детальні характеристики товарів (Specifications):
   - Товари різних категорій мають абсолютно різні атрибути (у ноутбука — процесор, у взуття — розмір). В SQL це призводить до створення таблиць з великою кількістю NULL полів або використання повільного патерну EAV
   - Перевага NoSQL полягає в тому що кожен товар зберігається як документ з унікальним набором полів
2. **Відгуки та контент користувачів (Reviews):**
   - Відгук є складною сутністю, що містить текст, рейтинг, масив фотографій та дерево відповідей/коментарів. У реляційній базі це вимагає створення мінімум 3-х таблиць та використання JOIN при кожному читанні

   - NoSQL: Можливість зберігати фото та відповіді як вкладені масиви (Arrays/Embedded Documents) всередині одного документа відгуку. Це значно пришвидшує читання (Read Heavy workload)

Для реалізації було створено базу даних ShopDB_NoSQL та дві колекції.

Колекція 1: product_specs зберігає динамічні характеристики товару. Використовується посилання ref_product_id на основну SQL таблицю

Колекція 2: product_reviews реалізовано вкладеність даних, яка неможлива в плоских таблицях SQL без додаткових зв'язків

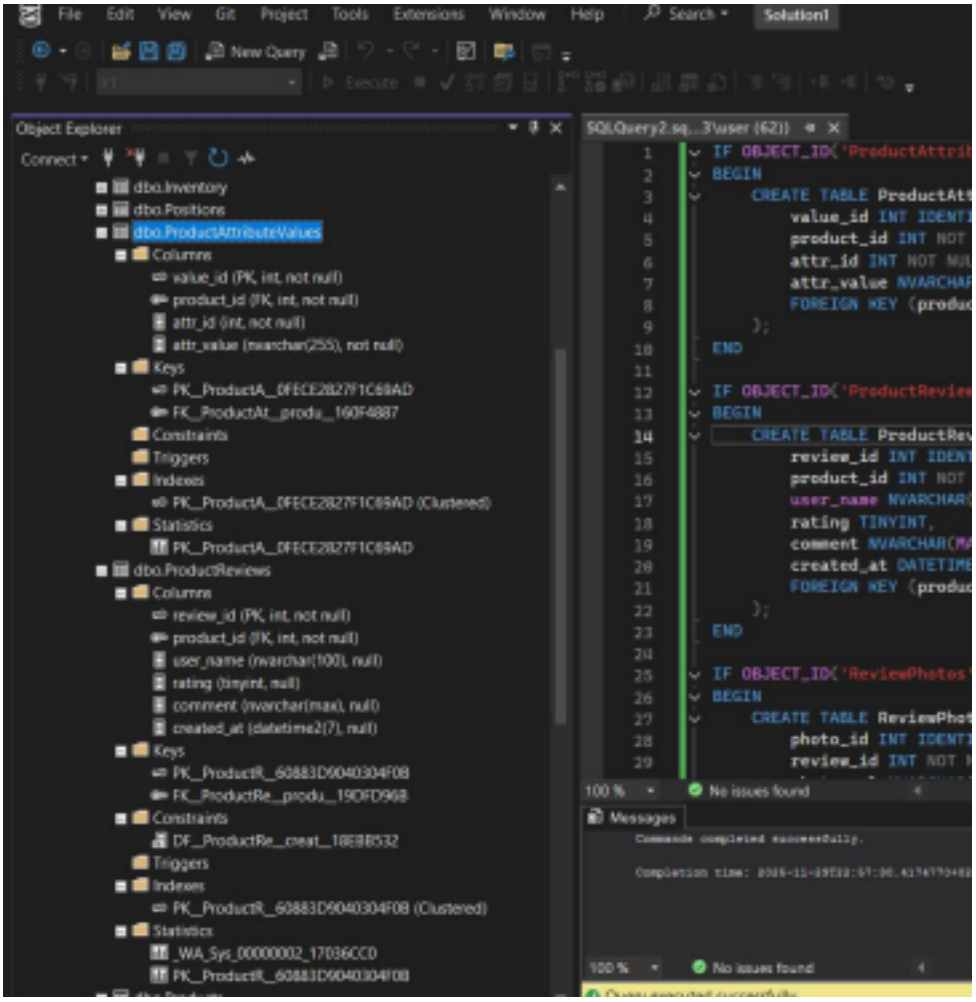


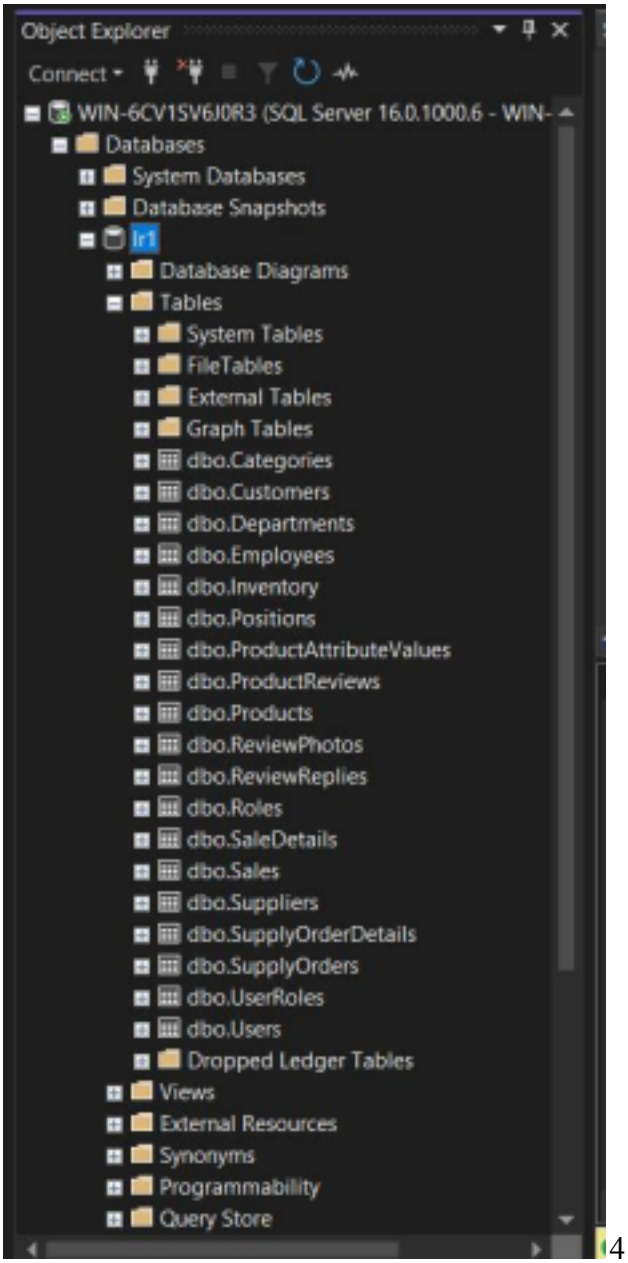
Для коректного порівняння швидкодії, аналогічна структура даних була реалізована в реляційній базі. Це призвело до нормалізації даних та збільшення кількості таблиць

Для відображення структури відгуків в SQL було створено **3 взаємопов'язані таблиці**:

1. ProductReviews — основна інформація (текст, рейтинг)
2. ReviewPhotos — зберігання посилань на фото (зв'язок 1-до-багатьох)
3. ReviewReplies — зберігання відповідей (зв'язок 1-до-багатьох)

Також використано таблицю ProductAttributeValues для реалізації EAV-патерну характеристик


File Edit View Git Project Tools Extensions Window Help
Search
Solution1
New Query
Execute
Object Explorer
Connect
dbo.Inventory
dbo.Positions
dbo.ProductAttributeValues
Columns
value_id (PK, int, not null)
product_id (FK, int, not null)
attr_id (int, not null)
attr_value (nvarchar(255), not null)
Keys
PK__ProductA__0FECE2827F1C89AD
FK__ProductAt__produ__160F4887
Constraints
Triggers
Indexes
PK__ProductA__0FECE2827F1C89AD (Clustered)
Statistics
PK__ProductA__0FECE2827F1C89AD
dbo.ProductReviews
Columns
review_id (PK, int, not null)
product_id (FK, int, not null)
user_name (nvarchar(100), null)
rating (tinyint, null)
comment (nvarchar(max), null)
created_at (datetime2(7), null)
Keys
PK__ProductR__60883D9040304F08
FK__ProductRe__produ__19DFD96B
Constraints
DF__ProductRe__creat__18EBB532
Triggers
Indexes
PK__ProductR__60883D9040304F08 (Clustered)
Statistics
_WA_Sys_00000002_17036CC0
PK__ProductR__60883D9040304F08
SQLQuery2.sq...3\user (62))
IF OBJECT_ID('ProductAttri
BEGIN
CREATE TABLE ProductAtt
value_id INT IDENTI
product_id INT NOT
attr_id INT NOT NUL
attr_value NVARCHAR
FOREIGN KEY (produc
);
END
IF OBJECT_ID('ProductReview
BEGIN
CREATE TABLE ProductRev
review_id INT IDENT
product_id INT NOT
user_name NVARCHAR
rating TINYINT,
comment NVARCHAR(MA
created_at DATETIME
FOREIGN KEY (produc
);
END
IF OBJECT_ID('ReviewPhotos'
BEGIN
CREATE TABLE ReviewPhot
photo_id INT IDENTI
review_id INT NOT
100 %
No issues found
Messages
Commands completed successfully.
Completion time:
100 %
No issues found



4

Для перевірки ефективності було розроблено скрипт на мові **Python**, який автоматизував процес тестування:

1. Генерація даних: Було створено 2000 товарів. В обидві бази (SQL та NoSQL) записано ідентичну інформацію (характеристики, відгуки, фото).
2. Сценарій тесту: Отримання повної сторінки відгуків (з фото та відповідями) для 500 випадкових товарів.

Порівняння запитів:

SQL Запит: Використовує множинні LEFT JOIN для об'єднання таблиць ProductReviews, ReviewPhotos та ReviewReplies. Це створює велике навантаження на процесор СУБД.

NoSQL Запит: Використовує простий пошук по індексу (find), оскільки всі дані вже знаходяться в одному документі (попередньо згруповані)

Тестування показало, що MongoDB обробляє запити на читання складної структури в **1,9** разів швидше, ніж MS SQL Server. Це пояснюється відсутністю необхідності виконувати дорогі операції з'єднання таблиць (JOIN) та денормалізованою природою документоорієнтованих баз даних